

#1. 생산자- 소비자로 구성된 응용 프로그램 만들기

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#include <time.h>
```

```
#define BUFFER_SIZE 4
```

```
typedef struct {
```

```
    int buffer[BUFFER_SIZE]; // 순환 버퍼 배열
```

```
    int in; // 다음 쓰기 작업을 위한 인덱스
```

```
    int out; // 다음 읽기 작업을 위한 인덱스
```

```
} CircularQueue;
```

```
CircularQueue queue;
```

```
sem_t semWrite; // 쓰기 가능한 슬롯을 추적하기 위한 세마포어
```

```
sem_t semRead; // 읽기 가능한 항목을 추적하기 위한 세마포어
```

```
pthread_mutex_t mutex; // 공유 버퍼 접근을 보호하기 위한 뮤텍스
```

```
/**
```

```
 * @brief 항목을 순환 버퍼에 추가합니다.
```

```
 *
```

```
 * @param item 버퍼에 추가할 항목입니다.
```

```
 */
```

```
void enqueue(int item) {
```

```
    queue.buffer[queue.in] = item; // 항목을 버퍼에 저장
```

```
    queue.in = (queue.in + 1) % BUFFER_SIZE; // 순환 방식으로 쓰기 인덱스 업데이트
```

```
producer : wrote 0
consumer : read 0
producer : wrote 1
consumer : read 1
producer : wrote 2
producer : wrote 3
producer : wrote 4
producer : wrote 5
consumer : read 2
consumer : read 3
producer : wrote 6
consumer : read 4
consumer : read 5
consumer : read 6
producer : wrote 7
producer : wrote 8
consumer : read 7
consumer : read 8
producer : wrote 9
consumer : read 9
```

```

}

/**
 * @brief 순환 버퍼에서 항목을 제거합니다.
 *
 * @return int 버퍼에서 제거된 항목입니다.
 */
int dequeue() {
    int item = queue.buffer[queue.out]; // 버퍼에서 항목을 가져옴
    queue.out = (queue.out + 1) % BUFFER_SIZE; // 순환 방식으로 읽기 인덱스 업데이트
    return item;
}

```

```

/**
 * @brief 항목을 생산하고 버퍼에 쓰는 생산자 스레드 함수입니다.
 *
 * @param param 사용되지 않는 매개변수입니다.
 * @return void*
 */
void *producer(void *param) {
    for (int i = 0; i < 10; i++) {
        sleep(rand() % 3); // 0에서 2초 사이의 랜덤 대기 시간

        int item = i; // 생산할 항목
        sem_wait(&semWrite); // 버퍼에 사용 가능한 슬롯이 있을 때까지 대기
        pthread_mutex_lock(&mutex); // 버퍼 접근 잠금

        enqueue(item); // 항목을 버퍼에 추가
        printf("producer : wrote %d\n", item);
    }
}

```

```

    pthread_mutex_unlock(&mutex); // 버퍼 접근 잠금 해제

    sem_post(&semRead); // 버퍼에 새로운 항목이 있음을 신호
}

pthread_exit(0); // 스레드 종료
}

/**
 * @brief 버퍼에서 항목을 소비하는 소비자 스레드 함수입니다.
 *
 * @param param 사용되지 않는 매개변수입니다.
 * @return void*
 */
void *consumer(void *param) {
    for (int i = 0; i < 10; i++) {
        sleep(rand() % 3); // 0에서 2초 사이의 랜덤 대기 시간

        sem_wait(&semRead); // 버퍼에 사용 가능한 항목이 있을 때까지 대기
        pthread_mutex_lock(&mutex); // 버퍼 접근 잠금

        int item = dequeue(); // 버퍼에서 항목 제거
        printf("consumer : read %d\n", item);

        pthread_mutex_unlock(&mutex); // 버퍼 접근 잠금 해제
        sem_post(&semWrite); // 버퍼에 사용 가능한 슬롯이 있음을 신호
    }

    pthread_exit(0); // 스레드 종료
}

```

```
int main() {

    pthread_t producerThread, consumerThread;

    srand(time(NULL)); // 랜덤 숫자 생성기 시드 설정

    queue.in = 0; // 쓰기 인덱스 초기화
    queue.out = 0; // 읽기 인덱스 초기화

    sem_init(&semWrite, 0, BUFFER_SIZE); // 쓰기 세마포어 초기화 (BUFFER_SIZE로 설정)
    sem_init(&semRead, 0, 0); // 읽기 세마포어 초기화 (0으로 설정)
    pthread_mutex_init(&mutex, NULL); // 뮤텝스 초기화

    pthread_create(&producerThread, NULL, producer, NULL); // 생산자 스레드 생성
    pthread_create(&consumerThread, NULL, consumer, NULL); // 소비자 스레드 생성

    pthread_join(producerThread, NULL); // 생산자 스레드 종료 대기
    pthread_join(consumerThread, NULL); // 소비자 스레드 종료 대기

    sem_destroy(&semWrite); // 쓰기 세마포어 제거
    sem_destroy(&semRead); // 읽기 세마포어 제거
    pthread_mutex_destroy(&mutex); // 뮤텝스 제거

    return 0;

}
```

#2. 소프트웨어로 문을 만드는 방법

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <unistd.h>
```

```
#include <stdatomic.h>
```

```
#include <time.h>
```

```
#define BUFFER_SIZE 4
```

```
#define MAX_MESSAGES 20
```

```
int buffer[BUFFER_SIZE];
```

```
int in = 0;
```

```
int out = 0;
```

```
int next_value = 0;
```

```
/**
```

```
 * @brief 세마포어 구조체 정의
```

```
 *
```

```
 * _Atomic은 stdatomic.h 라이브러리에서 제공하는 원자적 연산을 위한 키워드입니다.
```

```
 * 이 라이브러리는 멀티스레딩 환경에서 변수의 원자적 조작을 보장합니다.
```

```
 */
```

```
typedef struct {
```

```
    _Atomic int value;
```

```
} Semaphore;
```

```
/**
```

```
 * @brief 메시지 구조체 정의
```

```
producer : wrote 0
consumer : read 0
producer : wrote 1
consumer : read 1
producer : wrote 2
consumer : read 2
producer : wrote 3
consumer : read 3
producer : wrote 4
consumer : read 4
producer : wrote 5
consumer : read 5
producer : wrote 6
consumer : read 6
producer : wrote 7
producer : wrote 8
consumer : read 7
producer : wrote 9
consumer : read 8
consumer : read 9
```

```

*/

typedef struct {

    char message[50];

} Message;


Message messages[MAX_MESSAGES];

int message_index = 0;


pthread_mutex_t message_mutex;


/**
 * @brief 세마포어를 초기화합니다.
 *
 * @param sem 초기화할 세마포어
 * @param value 세마포어의 초기 값
 */
void sem_init(Semaphore *sem, int value) {

    atomic_init(&sem->value, value); // 원자적 초기화

}


/**
 * @brief 세마포어를 기다립니다. (세마포어 값을 1 감소)
 *
 * @param sem 대기할 세마포어
 */
void sem_wait(Semaphore *sem) {

    while (1) {

        int expected = atomic_load(&sem->value); // 세마포어 값 원자적 읽기

        while (expected <= 0) { // 세마포어 값이 0 이하일 때 대기

```

```

        expected = atomic_load(&sem->value);

    }

    if (atomic_compare_exchange_weak(&sem->value, &expected, expected - 1)) { // 원자적 비교 후 값 변
경
        break;

    }

}

}

```

```

/**
 * @brief 세마포어를 신호합니다. (세마포어 값을 1 증가)
 *
 * @param sem 신호할 세마포어
 */
void sem_signal(Semaphore *sem) {
    atomic_fetch_add(&sem->value, 1); // 세마포어 값 원자적 증가
}

```

```

Semaphore semWrite; // 쓰기 세마포어
Semaphore semRead;  // 읽기 세마포어

```

```

int flag[2] = {0, 0};

int turn;

```

```

#define barrier() asm volatile("mfence" ::: "memory")

```

```

/**
 * @brief 진입 임계 구역
 *

```

```

* @param self 자기 자신의 스레드 번호
*/

void enter_critical_section(int self) {

    barrier(); // 메모리 배리어 추가

    flag[self] = 1;

    turn = 1 - self;

    while (flag[1 - self] == 1 && turn == 1 - self);

    barrier(); // 메모리 배리어 추가

}

/**
* @brief 탈출 임계 구역
*
* @param self 자기 자신의 스레드 번호
*/

void leave_critical_section(int self) {

    barrier(); // 메모리 배리어 추가

    flag[self] = 0;

    barrier(); // 메모리 배리어 추가

}

/**
* @brief 항목을 생산하고 버퍼에 쓰는 생산자 스레드 함수
*
* @param param 사용되지 않는 매개변수
* @return void*
*/

void *producer(void *param) {

    int self = 0;

    for (int i = 0; i < 10; i++) {

```



```

sem_wait(&semWrite);

enter_critical_section(self);


buffer[in] = next_value;

next_value = (next_value + 1) % 10;

pthread_mutex_lock(&message_mutex);

snprintf(messages[message_index++].message, 50, "producer : wrote %d", buffer[in]);

pthread_mutex_unlock(&message_mutex);

in = (in + 1) % BUFFER_SIZE;


leave_critical_section(self);

sem_signal(&semRead);

sleep(rand() % 3); // 0에서 2초 사이의 랜덤 대기 시간

}

pthread_exit(0); // 스레드 종료

}

/**
 * @brief 버퍼에서 항목을 소비하는 소비자 스레드 함수
 *
 * @param param 사용되지 않는 매개변수
 * @return void*
 */
void *consumer(void *param) {

    int self = 1;

    for (int i = 0; i < 10; i++) {

        sem_wait(&semRead);

        enter_critical_section(self);

        int item = buffer[out];

        pthread_mutex_lock(&message_mutex);

```

```

    snprintf(messages[message_index++].message, 50, "consumer : read %d", item);

    pthread_mutex_unlock(&message_mutex);

    out = (out + 1) % BUFFER_SIZE;

    leave_critical_section(self);

    sem_signal(&semWrite);

    sleep(rand() % 3); // 0에서 2초 사이의 랜덤 대기 시간
}

pthread_exit(0); // 스레드 종료
}

int main() {
    pthread_t tid1, tid2;

    srand(time(NULL)); // 난수 생성기 초기화

    pthread_mutex_init(&message_mutex, NULL); // 메시지 뮤텝스 초기화

    sem_init(&semWrite, BUFFER_SIZE); // 쓰기 세마포어 초기화

    sem_init(&semRead, 0); // 읽기 세마포어 초기화

    pthread_create(&tid1, NULL, producer, NULL); // 생산자 스레드 생성
    pthread_create(&tid2, NULL, consumer, NULL); // 소비자 스레드 생성

    pthread_join(tid1, NULL); // 생산자 스레드 종료 대기
    pthread_join(tid2, NULL); // 소비자 스레드 종료 대기

    for (int i = 0; i < message_index; i++) {
        printf("%s\n", messages[i].message); // 메시지 출력

        fflush(stdout); // 출력 버퍼 강제 플러시
    }

    pthread_mutex_destroy(&message_mutex); // 메시지 뮤텝스 제거

    return 0;
}

```

#3. 내 컴퓨터의 페이지 크기는 얼마일까? : getconf 명령어로 페이지 사이즈를 확인할 수 있음

```
minseo@minseo:~/Virtual_Shared$ time ./page
real    0m0.088s
user    0m0.079s
sys     0m0.009s
minseo@minseo:~/Virtual_Shared$ time ./page 1022
real    0m0.101s
user    0m0.090s
sys     0m0.009s
minseo@minseo:~/Virtual_Shared$ time ./page 1023
real    0m0.504s
user    0m0.484s
sys     0m0.010s
minseo@minseo:~/Virtual_Shared$ time ./page 1024
real    0m0.556s
user    0m0.531s
sys     0m0.020s
minseo@minseo:~/Virtual_Shared$ time ./page 1025
real    0m0.435s
user    0m0.413s
sys     0m0.020s
minseo@minseo:~/Virtual_Shared$ time ./page 1026
real    0m0.097s
user    0m0.096s
sys     0m0.000s
minseo@minseo:~/Virtual_Shared$ time ./page 2042
real    0m0.095s
user    0m0.083s
sys     0m0.011s
minseo@minseo:~/Virtual_Shared$ time ./page 2048
real    0m0.553s
user    0m0.539s
sys     0m0.010s
minseo@minseo:~/Virtual_Shared$ time ./page 2043
real    0m0.091s
user    0m0.081s
sys     0m0.009s
minseo@minseo:~/Virtual_Shared$ time ./page 2046
real    0m0.091s
user    0m0.080s
sys     0m0.009s
minseo@minseo:~/Virtual_Shared$ time ./page 2049
real    0m0.440s
user    0m0.428s
sys     0m0.010s
minseo@minseo:~/Virtual_Shared$ time ./page 2047
real    0m0.514s
user    0m0.491s
sys     0m0.020s
minseo@minseo:~/Virtual_Shared$ |
```

```
minseo@minseo:~/Virtual_Shared$ getconf PAGE_SIZE
4096
minseo@minseo:~/Virtual_Shared$ time ./page 4094
real    0m0.108s
user    0m0.096s
sys     0m0.011s
minseo@minseo:~/Virtual_Shared$ time ./page 4095
real    0m0.507s
user    0m0.494s
sys     0m0.010s
minseo@minseo:~/Virtual_Shared$ time ./page 4096
real    0m0.558s
user    0m0.525s
sys     0m0.029s
minseo@minseo:~/Virtual_Shared$ time ./page 4097
real    0m0.433s
user    0m0.421s
sys     0m0.010s
minseo@minseo:~/Virtual_Shared$ time ./page 4098
real    0m0.092s
user    0m0.083s
sys     0m0.008s
minseo@minseo:~/Virtual_Shared$ |
```

